

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ИНСТИТУТ НЕПРЕРЫВНОГО И ДИСТАНЦИОННОГО ОБРАЗОВАНИЯ

КАФЕДРА №44

ОЦЕНКА

ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Д.В. Куртяник
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Разработка программы с использованием макрокоманд

по дисциплине: Программирование на языках Ассемблера

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

В3441
номер группы

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2025

1 Цель лабораторной работы:

Изучение методики логической структуризации ассемблерной программы с помощью процедур, рассмотрение способов вызова процедур, изучение функциональных возможностей макроопределений, расширение представления о возможностях синтаксиса ассемблера.

2 Номер варианта и индивидуальное задание

Вариант 14: Дан массив чисел размером в слово. Проверить числа на чётность, заполняя массив с элементами размером в байт. Устанавливать значение 1, если число чётное; 0, если нечётное.

3 Описание выбранной модели решения, сопоставление и объяснение основных частей алгоритма и выбранных для их реализации синтаксических конструкций языка

3.1 Модель решения

Обход массива чисел (WORD), проверка каждого на чётность и запись результата в другой массив (BYTE), где значение 1 соответствует чётному числу, а 0 — нечётному.

3.2 Основные части алгоритма

3.2.1 Инициализация данных и регистров:

Объявлены массив numbers и результирующий массив result. Константа nLen определяет число элементов массива, вычисляется как деление общего размера массива на 2 (так как один элемент WORD занимает 2 байта). Регистр ESI указывает на начало исходного массива, а EDI — на начало массива результатов. Регистр ECX задаёт количество итераций (число элементов массива)

Использование цикла реализовано через инструкцию loop, которая уменьшает значение регистра ECX после каждой итерации и повторяет цикл до достижения нуля.

3.2.2 Проверка чётности

Каждое число из массива загружается в регистр AX через инструкцию `mov ax, WORD PTR [esi]`. Инструкция `test ax, 1` проверяет младший бит числа. Если младший бит равен 0, то число чётное, иначе — нечётное.

3.2.3 Запись результата

В зависимости от результата проверки выполняется запись 1 (если число чётное) или 0 (если нечётное) в ячейку результирующего массива. Указатели ESI и EDI сдвигаются соответственно: на 2 байта (WORD) и на 1 байт (BYTE) для перехода к следующему элементу.

3.3 Использование макрокоманд

Часть с проверкой чётности вынесена в отдельную функцию, таким образом, по сравнению с первой версией кода улучшена модульность, а также код стал компактнее.

3.4 Процедура

Использование процедур позволяет передать аргумент через стек, однако это увеличивает накладные расходы из-за вызова процедуры для каждого элемента

3.5 Макрос

Код макроса встраивается напрямую в место вызова, что не создает накладных расходов на `call/ret` и передачу параметров через стек. Макрос разворачивается в инструкции на каждой итерации, что увеличивает размер исполняемого кода.

4 Исходный код

4.1 Исходный код с использованием процедуры

```
.386  
.model flat, stdcall  
.stack 4096
```

```
.data
    numbers WORD 2, 2, 1, 4, 10 ; исходный массив чисел (WORD) - пример
входных данных
    nLen EQU ($ - numbers) / 2 ; количество элементов в массиве
    result BYTE nLen DUP(0) ; массив результатов (BYTE) – размер равен
числу элементов
```

```
.code
```

```
check_even PROC n1:WORD ; объявление процедуры
    test n1, 1 ; проверяем четность, используя операцию test: если младший
бит равен 0, число четное
    sete al ; устанавливаем результат в al

    ret
check_even ENDP
```

```
main PROC
    mov esi, OFFSET numbers ; начало исходного массива
    mov edi, OFFSET result ; начало массива результатов
    mov ecx, nLen ; количество итераций = число элементов массива

check_loop:
    push WORD PTR [esi] ; загружаем число из исходного массива на стек
    call proc_check_even ; вызов процедуры
    mov byte [edi], al ; записываем результат в массив результатов

    add esi, 2 ; переходим к следующему элементу (WORD = 2 байта)
    add edi, 1 ; следующий элемент результата (BYTE = 1 байт)
    loop check_loop ; повторяем цикл

main ENDP
end main
```

4.2 Исходный код с использованием макросов

```
.386
.model flat, stdcall
.stack 4096
```

.data

numbers WORD 2, 2, 1, 4, 10 ; исходный массив чисел (WORD) - пример
входных данных
nLen EQU (\$ - numbers) / 2 ; количество элементов в массиве
result BYTE nLen DUP(0) ; массив результатов (BYTE) – размер равен
числу элементов

.code

check_even MACRO n1 ; объявление макроса
test n1, 1 ; проверяем четность, используя операцию test: если младший
бит равен 0, число четное
sete al ; устанавливаем результат в al
ENDM

main PROC

mov esi, OFFSET numbers ; начало исходного массива
mov edi, OFFSET result ; начало массива результатов
mov ecx, nLen ; количество итераций = число элементов массива

check_loop:

mov ax, WORD PTR [esi] ; загружаем число из исходного массива
check_even ax ; вызов макроса
mov [edi], al ; записываем результат в массив результатов

add esi, 2 ; переходим к следующему элементу (WORD = 2 байта)
add edi, 1 ; следующий элемент результата (BYTE = 1 байт)
loop check_loop ; повторяем цикл

main ENDP

end main

5 Снимки экрана с анализом работы программы

5.1 Использование процедуры

CPU - main thread, module lab

00401010	BE 00204000	MOV ESI, lab.00402000
00401015	BF 0A204000	MOV EDI, lab.0040200A
0040101A	B9 05000000	MOV ECX, 5
0040101F	66:FF36	PUSH WORD PTR DS:[ESI]
00401022	E8 D9FFFFFF	CALL lab.00401000
00401027	8B07	MOV BYTE PTR DS:[EDI], AL
00401029	83C6 02	ADD ESI, 2
0040102C	83C7 01	ADD EDI, 1
0040102F	E2 EE	LOOPD SHORT lab.0040101F
00401031	0000	ADD BYTE PTR DS:[EAX], AL
00401033	0000	ADD BYTE PTR DS:[EAX], AL
00401035	0000	ADD BYTE PTR DS:[EAX], AL
00401037	0000	ADD BYTE PTR DS:[EAX], AL
00401039	0000	ADD BYTE PTR DS:[EAX], AL
00401000=lab.00401000		

Registers (FPU)

EAX	0019FFC0
ECX	00000005
EDX	00401010
EBX	0036D000
ESP	0019FF00
EBP	00401010
ESI	00402000
EDI	0040200A
EIP	00401022
C 0	ES 002B 32bit 0(FFFFFFFF)
P 1	CS 0023 32bit 0(FFFFFFFF)
A 0	SS 002B 32bit 0(FFFFFFFF)
Z 1	DS 002B 32bit 0(FFFFFFFF)
S 0	FS 0053 32bit 370000(FFF)
T 0	GS 002B 32bit 0(FFFFFFFF)

начало точно такое же как в первой версии кода
затем происходит вызов процедуры, который кладет текущее значение на стек

Address	Hex dump	ASCII
00402000	02 00 02 00 01 00 04 00	0.0.0.0
00402008	0A 00 00 00 00 00 00 00	
00402010	00 00 00 00 00 00 00 00	
00402018	00 00 00 00 00 00 00 00	
00402020	00 00 00 00 00 00 00 00	

0019FF72 00000002

CPU - main thread, module lab

00401000	55	PUSH EBP
00401001	8BEC	MOV EBP, ESP
00401003	66:F745 08 01	TEST WORD PTR SS:[EBP+8], 1
00401009	0F94C0	SETL AL
0040100C	C9	LEAVE
0040100D	C2 0400	RETN 4
00401010	BE 00204000	MOV ESI, lab.00402000
00401015	BF 0A204000	MOV EDI, lab.0040200A
0040101A	B9 05000000	MOV ECX, 5
0040101F	66:FF36	PUSH WORD PTR DS:[ESI]
00401022	E8 D9FFFFFF	CALL lab.00401000
00401027	8B07	MOV BYTE PTR DS:[EDI], AL
00401029	83C6 02	ADD ESI, 2
0040102C	83C7 01	ADD EDI, 1

Registers (FPU)

EAX	0019FFC0
ECX	00000005
EDX	00401010
EBX	0036D000
ESP	0019FF00
EBP	0019FF00
ESI	00402000
EDI	0040200A
EIP	00401000
C 0	ES 002B 32bit 0(FFFFFFFF)
P 1	CS 0023 32bit 0(FFFFFFFF)
A 0	SS 002B 32bit 0(FFFFFFFF)
Z 1	DS 002B 32bit 0(FFFFFFFF)
S 0	FS 0053 32bit 370000(FFF)
T 0	GS 002B 32bit 0(FFFFFFFF)

затем происходит переход к процедуре

Address	Hex dump	ASCII
00402000	02 00 02 00 01 00 04 00	0.0.0.0
00402008	0A 00 00 00 00 00 00 00	
00402010	00 00 00 00 00 00 00 00	
00402018	00 00 00 00 00 00 00 00	
00402020	00 00 00 00 00 00 00 00	

0019FF6E 00401027 RETURN to lab.00401027 from lab.00401000

CPU - main thread, module lab

00401000	55	PUSH EBP
00401001	8BEC	MOV EBP, ESP
00401003	66:F745 08 01	TEST WORD PTR SS:[EBP+8], 1
00401009	0F94C0	SETL AL
0040100C	C9	LEAVE
0040100D	C2 0400	RETN 4
00401010	BE 00204000	MOV ESI, lab.00402000
00401015	BF 0A204000	MOV EDI, lab.0040200A
0040101A	B9 05000000	MOV ECX, 5
0040101F	66:FF36	PUSH WORD PTR DS:[ESI]
00401022	E8 D9FFFFFF	CALL lab.00401000
00401027	8B07	MOV BYTE PTR DS:[EDI], AL
00401029	83C6 02	ADD ESI, 2
0040102C	83C7 01	ADD EDI, 1

Registers (FPU)

EAX	0019FFC0
ECX	00000005
EDX	00401010
EBX	0036D000
ESP	0019FF00
EBP	0019FF00
ESI	00402000
EDI	0040200A
EIP	00401003
C 0	ES 002B 32bit 0(FFFFFFFF)
P 1	CS 0023 32bit 0(FFFFFFFF)
A 0	SS 002B 32bit 0(FFFFFFFF)
Z 1	DS 002B 32bit 0(FFFFFFFF)
S 0	FS 0053 32bit 370000(FFF)
T 0	GS 002B 32bit 0(FFFFFFFF)

в процедуре выполняется добавленный компилятором код он сохраняет базовый адрес стекового кадра

Address	Hex dump	ASCII
00402000	02 00 02 00 01 00 04 00	0.0.0.0
00402008	0A 00 00 00 00 00 00 00	
00402010	00 00 00 00 00 00 00 00	
00402018	00 00 00 00 00 00 00 00	
00402020	00 00 00 00 00 00 00 00	

0019FF60 0019FF00 RETURN to lab.00401027 from lab.00401000

CPU - main thread, module lab

00401000	55	PUSH EBP
00401001	8BEC	MOV EBP, ESP
00401003	66:F745 08 01	TEST WORD PTR SS:[EBP+8], 1
00401009	0F94C0	SETL AL
0040100C	C9	LEAVE
0040100D	C2 0400	RETN 4
00401010	BE 00204000	MOV ESI, lab.00402000
00401015	BF 0A204000	MOV EDI, lab.0040200A
0040101A	B9 05000000	MOV ECX, 5
0040101F	66:FF36	PUSH WORD PTR DS:[ESI]
00401022	E8 D9FFFFFF	CALL lab.00401000
00401027	8B07	MOV BYTE PTR DS:[EDI], AL
00401029	83C6 02	ADD ESI, 2
0040102C	83C7 01	ADD EDI, 1

Registers (FPU)

EAX	0019FF01
ECX	00000005
EDX	00401010
EBX	0036D000
ESP	0019FF00
EBP	0019FF00
ESI	00402000
EDI	0040200A
EIP	0040100C
C 0	ES 002B 32bit 0(FFFFFFFF)
P 1	CS 0023 32bit 0(FFFFFFFF)
A 0	SS 002B 32bit 0(FFFFFFFF)
Z 1	DS 002B 32bit 0(FFFFFFFF)
S 0	FS 0053 32bit 370000(FFF)
T 0	GS 002B 32bit 0(FFFFFFFF)

выполняется проверка четности и сохранение результата в al

Address	Hex dump	ASCII
00402000	02 00 02 00 01 00 04 00	0.0.0.0
00402008	0A 00 00 00 00 00 00 00	
00402010	00 00 00 00 00 00 00 00	
00402020	00 00 00 00 00 00 00 00	

0019FF60 0019FF00 RETURN to lab.00401027 from lab.00401000

CPU - main thread, module lab

00401000	55	PUSH EBP
00401001	8BEC	MOV EBP, ESP
00401003	66:F745 08 01	TEST WORD PTR SS:[EBP+8], 1
00401009	0F94C0	SETL AL
0040100C	C9	LEAVE
0040100D	C2 0400	RETN 4
00401010	BE 00204000	MOV ESI, lab.00402000
00401015	BF 0A204000	MOV EDI, lab.0040200A
0040101A	B9 05000000	MOV ECX, 5
0040101F	66:FF36	PUSH WORD PTR DS:[ESI]
00401022	E8 D9FFFFFF	CALL lab.00401000
00401027	8B07	MOV BYTE PTR DS:[EDI], AL
00401029	83C6 02	ADD ESI, 2
0040102C	83C7 01	ADD EDI, 1

Registers (FPU)

EAX	0019FF01
ECX	00000005
EDX	00401010
EBX	0036D000
ESP	0019FF00
EBP	0019FF00
ESI	00402000
EDI	0040200A
EIP	00401027
C 0	ES 002B 32bit 0(FFFFFFFF)
P 1	CS 0023 32bit 0(FFFFFFFF)
A 0	SS 002B 32bit 0(FFFFFFFF)
Z 1	DS 002B 32bit 0(FFFFFFFF)
S 0	FS 0053 32bit 370000(FFF)
T 0	GS 002B 32bit 0(FFFFFFFF)

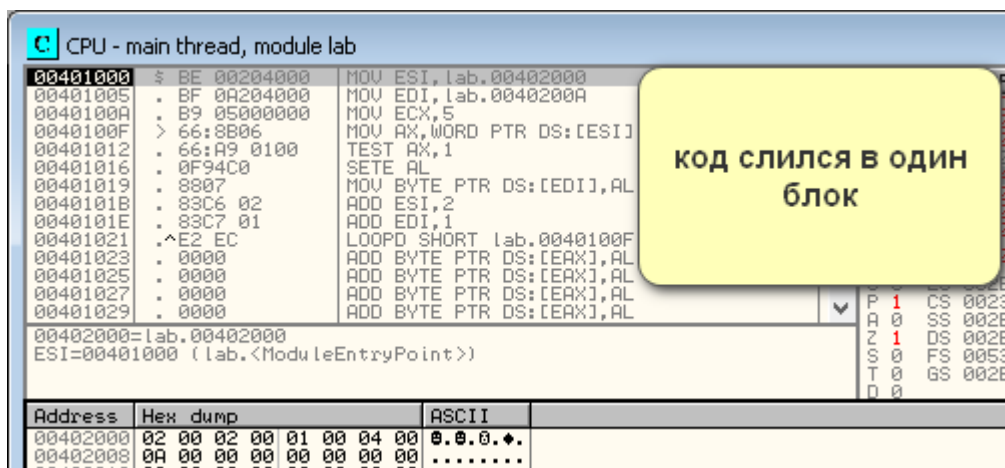
а также возврат к основной программе

Address	Hex dump	ASCII
00402000	02 00 02 00 01 00 04 00	0.0.0.0
00402008	0A 00 00 00 00 00 00 00	
00402010	00 00 00 00 00 00 00 00	
00402018	00 00 00 00 00 00 00 00	
00402020	00 00 00 00 00 00 00 00	

0019FF76 000076F2

Далее выполняется сохранение результата в память через `al` и повторение логики до обнуления счетчика.

5.2 Использование макроса



Логика осталась старой

6 Вывод

Для текущей задачи макрос является более предпочтительным. Он дает максимальную производительность и минимизируют накладные расходы. Процедура здесь избыточна и замедляет выполнение из-за частых вызовов и работы со стеком.